

PROPOSED SOLUTION: AI-POWERED E-COMMERCE PLATFORM

Database-Grounded Customer Assistance and Modular Business Operations

Abstract - The proposed system is a modular e-commerce platform for small and medium retailers that combines standard online shopping workflows with local AI assistance. A React/Vite client communicates with a Django REST backend containing account, catalog, cart, order, payment, inventory and chat modules. Product questions are answered from verified database records before the request is passed to the local Ollama model, reducing fabricated prices or product claims. The design emphasizes low operating cost, clear permissions, maintainable APIs and gradual integration within one semester.

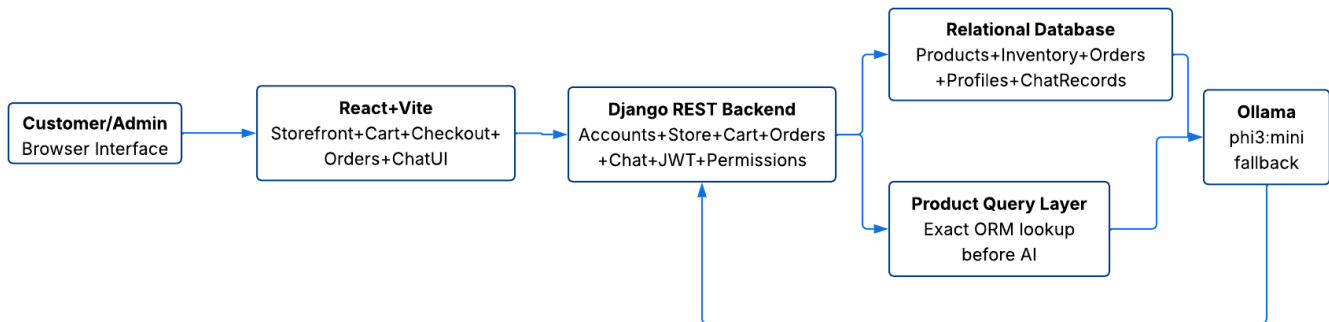


Fig. 1. Proposed client, backend, database and local-AI interaction flow.

I. PROPOSED ARCHITECTURE

The client layer uses React and Vite for product browsing, category navigation, registration, login, cart, checkout, order history, profile and chat pages. **The service layer** uses Django 4.2 and Django REST Framework. Existing backend modules expose REST resources for profiles, categories, products, inventory, carts, cart items, orders, payments, invoices and chat records. JWT endpoints provide token-based authentication, while role and object permissions restrict protected operations. **The data layer** uses relational models so product, stock, transaction and conversation records remain consistent. Docker-based environment packaging is planned for reproducible deployment.

II. CUSTOMER AND ADMIN WORKFLOWS

A customer registers or logs in, browses active products, adds items to a cart, completes checkout and reviews previous orders. Public users may read catalog information, while authenticated users perform account and transaction operations. Administrators manage categories, products, prices, stock and order status through Django Admin and protected APIs. Inventory changes are recorded as additions or deductions instead of silently replacing history. Human-support chat records are retained separately from automated responses so manual takeover can be added without redesigning the database.

III. DATABASE-GROUNDED AI ASSISTANCE

The customer chatbot endpoint is **POST /api/ai/chat/**. When a message arrives, a deterministic query layer first checks whether it concerns products. It searches only active, in-stock Product records and supports listing available items; filtering below, above or between prices; and returning one product's name, price, short description and stock. If no product intent is detected, the message is sent to the locally installed **Ollama phi3:mini** model. The language model never receives permission to modify the database. This hybrid flow provides exact commercial data while preserving general conversational assistance.

IV. DATA INTEGRITY AND SECURITY

Prices use decimal fields to avoid floating-point monetary errors. Products belong to categories, cart items belong to carts, order items belong to orders, and inventory logs reference both products and authorized modifiers. Server-side serializers validate API input. JWT authentication, read-only public catalog access and support/admin permissions protect sensitive resources. Secrets, virtual environments, node modules and local SQLite databases are excluded from version control.

V. CURRENT PROTOTYPE EVIDENCE

The integrated prototype currently runs the React client at **localhost:5173** and Django at **127.0.0.1:8000**. Five sample electronic products were entered through Django Admin. PowerShell and browser tests confirmed that the chatbot lists all products, returns products under Tk 2000 and reports Wireless Mouse price, description and stock. The same responses were verified through the React chat interface, and **python manage.py check** reported no system issues. This work is recorded in commit **d42c0ae**.

VI. COMPLETION PLAN AND EXPECTED RESULT

Next work will integrate the latest team branches, test Rahat's APIs through the browsable API and Postman, create reusable product seed data, resolve the pending accounts migration, improve frontend API integration, and implement the planned admin analytics and human-takeover features. The expected result is a low-cost storefront where customers receive reliable product guidance and merchants can manage catalog, inventory, orders and support from one maintainable platform.

VII. VALIDATION AND FEASIBILITY

Validation will combine browser-based API checks, Postman requests, permission tests and end-to-end frontend scenarios. Success criteria are correct HTTP status codes, exact agreement between chatbot product answers and database values, rejection of unauthorized modifications, and completion of cart-to-order workflows without inconsistent totals. Local Ollama inference avoids external API fees and keeps demonstration data on the development machine. The modular Django apps and separate Git branches also allow members to work independently before controlled integration.

REFERENCES

- [1] Group 1, "AI-Powered E-Commerce Platform with Intelligent Business Insights," CSE299 Project Proposal, 2026.
- [2] Django Software Foundation, "Django Documentation," docs.djangoproject.com.
- [3] Django REST Framework, "API Guide," django-rest-framework.org.
- [4] Ollama, "Local Model Runtime and API Documentation," ollama.com.
- [5] OpenAI, "ChatGPT," drafting and technical-assistance tool, accessed July 2026.